

Data Engineer Technical Case Study

HOMA GAMES



Objective

Build a small, production-minded analytics layer from raw mobile game events so that analysts can compute retention and level balancing metrics.

You are **encouraged to use GenAI / coding agents**. We will evaluate **correctness, structure, and engineering judgment**, not code volume.

We care about correctness, clarity of assumptions, and whether your data model makes analyst work easy and safe. We do not care about framework choice or visual polish.

Read through the whole document to understand all the requirements and desired artifacts before starting implementation.

Timebox: 3 to 4 hours (please do not exceed).

Dataset & Raw Schema

You can access 2 hive partitioned data sets for the month of November 2025 containing player activity events:

- **Install events** (triggered when a player installs the game)
- **Level progression events** (only the following types are relevant for the exercise):
 - Level_started (triggered when a player starts a level)
 - Level_completed (triggered when a player successfully completes a level)
 - Level_failed (triggered when a player fails to complete a level)

The data is available at URLs:

- Installs:
https://hg-data-interview.s3.us-east-2.amazonaws.com/data_eng_202512_Inqefimc/installs/d_event_received%3DYYYY-MM-DD/bc6c84620f8347038387981545e35736-0.parquet - with YYYY-MM-DD being the date
- levels:
http://hg-data-interview.s3.us-east-2.amazonaws.com/data_eng_202512_Inqefimc/levels/d_event_received%3DYYYY-MM-DD/383e3756cd8f44939fe742dc391a6939-0.parquet - with YYYY-MM-DD being the date

Data Engineer Technical Case Study

HOMA GAMES



Data schema

- d_event_received (date) — date the event was received (partition key)
- i_timestamp (int) — **unix timestamp** of event reception
- s_user_id (string) — user id
- s_country_code (string) — country the event was triggered from
- s_event_client_timestamp (string) — client-side event timestamp (UTC)
- s_event_name (string) — name of the event
- s_event_value (string) — payload (may contain level id and other event specific attributes)

Events **can be received at a different time / date than when they were triggered** – this typically happens when a player plays offline and the event data gets sent to our servers when the player comes back online. Choose a canonical time for your metrics and justify this choice.

1) Ingest and normalize events

Create a clean/staged representation of the events with:

- Consistent timestamp parsing
- Basic type normalization
- Idempotent re-runs

You may implement using any locally runnable stack (Python + SQL, DuckDB, dbt, pandas, Spark, etc.). Keep it lightweight and reproducible.

2) Produce analytics-ready datasets

Create tables/views (or equivalent outputs) that make metric computation easy, efficient and safe for analysts.

The metrics and dimensions you need to support:

- Install count (dimensions: country, install date)
- Levels started, completed, failed (dimensions: country, event date, level id)
- Retention D1, D7 (dimensions: country, install date)
- Level win rate (dimensions: country, install date, level id)

Data Engineer Technical Case Study

HOMA GAMES



Retention definition: a user is considered “retained at DX” if they start/finish at least a level on their install date + X days (based on UTC calendar days).

Install date attribution note: Some level events will belong to users whose install is not observable in the dataset window. You should explicitly decide how to handle such events (exclude, approximate, or define a cohorting rule), and document the implications.

3) Derive metrics

Provide a notebook or queries that use the previous datasets to produce the following results:

- a) Average installs per day for each country during the month of November
- b) Top 5 lowest win rate levels with their win rate
- c) Win rate of level 25 for users that installed on the 2nd of November
- d) The retention D7 for French users that installed on the 11th of November
- e) The trailing 7 day moving average of installs per day for the whole month of November

4) Bonus points

If you have time you can make sure:

- Your pipeline supports incremental ingestion of new partitions without full recreation of all the tables
 - You have a good data quality report over the ingested/processed datasets
-

Deliverables

Provide a zip containing:

1. README

- How to run locally (exact commands)
- Assumptions and trade-offs you made during implementation
- A short description of your data model

2. Code

- Ingestion + transformation
- Metric notebook or queries
- Data quality report / notebook if you performed any